Yang Zhao

857-225-1826 • yangzhao200124@gmail.com • linkedin.com/in/yang-zhao-48b12431a • github.com/YZhao-prog

EDUCATION

Northeastern University — Boston, MA
Master of Science in Information Systems, GPA: 3.8/4.0 — Sep. 2024 – Dec. 2026 (Expected)
Coursework: Linux/UNIX Systems, Data Structures, Algorithms, Object-Oriented Design, High-Performance Computing for AI

The Chinese University of Hong Kong, Shenzhen — Shenzhen, China
Bachelor of Engineering in Computer Science and Engineering — Sep. 2020 – Jun. 2024
Coursework: Database Systems, Distributed and Parallel Computing, Operating Systems, Computer Architecture, Compilers, Computer Networks

University of California, Berkeley — Berkeley, CA
Summer Session (CS 61A: Structure and Interpretation of Computer Programs), GPA: 4.0/4.0 — May 2022 – Aug. 2022

PROFESSIONAL EXPERIENCE

Splunk — San Jose, CA
Backend Software Engineer Intern — Jun. 2026 – Sep. 2026

- Designed and implemented a **C++** cache management system for a **Raft-coordinated distributed search cluster**, introducing **size-aware eviction** that keeps local disk under a configurable ceiling, preventing disk-full failures while preserving search result availability.
- Built an **in-memory storage tracker**, replacing expensive **full-directory scans** with **O(1)** incremental disk usage accounting and eliminating repeated filesystem traversal.
- Engineered an artifact offloading pipeline across concurrent workers, **AWS S3**, and a **PostgreSQL-backed KV store**, using readiness checks, conditional writes, and durable retries to sustain **500 concurrent offloads** with **zero metadata inconsistencies** or duplicate uploads.
- Decoupled local cache cleanup from remote artifact expiration by refactoring the reaper architecture into separate owners, adding **paginated** KV row fetches and **batched record deletion** so expiration runs in bounded memory with fewer round trips.
- Instrumented the cache eviction pipeline with **before/after rollout metrics**, including **cache hit/miss** rate and S3 artifact restore latency, validating through a soak test of **125K+ searches** that eviction held local disk at its **4 GiB** ceiling on **100%** of cycles with **zero errors or crashes**.

Splunk — Boston, MA
Backend Software Engineer Intern — Sep. 2025 – Dec. 2025

- Built a cross-node artifact sharing system in **C++** for a **Raft-coordinated** distributed search cluster, persisting search results to **AWS S3** so any node can serve them via **REST APIs**.
- Shipped remote artifact storage behind a **feature flag** and supported its staged **rollout** from internal clusters to the full production fleet.
- Optimized the artifact storage pipeline with **asynchronous** uploads, keeping search latency unaffected under high write volume, and quantified the storage cost impact at **0.62%** through cost analysis.
- Delivered **zero-downtime** cluster upgrades by designing a **backward-compatible storage schema**, enabling **rolling migrations** across **multi-node clusters** without service interruption.
- Created a **pytest** suite covering artifact lifecycle and **REST API** behavior for **CI/CD** regression coverage, and load-tested the feature at a production scale of **600K+** daily searches with **zero failures**.

PROJECTS

SQL Database Engine | Rust

- Built a **SQL database engine** from scratch with a hand-written query parser, rule-based optimizer, and parallel execution engine, achieving **2.1×** throughput over a single-threaded baseline on analytical queries.
- Implemented a custom **LSM-tree** storage engine with memtables, SSTables, leveled compaction, and Bloom filters behind a pluggable engine interface, sustaining **500K QPS** on storage-layer point reads.
- Designed **MVCC** with **snapshot isolation** and optimistic concurrency control, accelerating transaction startup with an in-memory active-transaction cache and validating isolation semantics against dirty, non-repeatable, and phantom reads.

Distributed Key-Value Store with Raft Consensus | Go, RPC

- Implemented the **Raft consensus protocol** with leader election, log replication, persistence, snapshot compaction, and crash recovery, ensuring correct operation under network partitions.
- Built linearizable Key-Value service on **Raft**, supporting concurrent clients with request deduplication and at-most-once semantics, and validated correctness using linearizability testing.
- Developed **sharded Key-Value store** with replicated Shard Controller, supporting dynamic shard migration and rebalancing across Raft-backed replica groups while preserving consistency.

TECHNICAL SKILLS

Languages: C++, Rust, Go, Java, Python, SQL, JavaScript, TypeScript
Backend & APIs: Spring Boot, Django, FastAPI, Flask, Node.js, Express.js, gRPC, Protocol Buffers, RESTful APIs, GraphQL, WebSockets
Frontend: React, Next.js, Angular, Redux, Tailwind CSS, HTML, CSS
GPU & Parallel Computing: CUDA, cuDNN, NCCL, OpenMP, OpenACC, MPI, SIMD, PyTorch
Databases & Messaging: PostgreSQL, MySQL, MongoDB, Redis, RocksDB, Elasticsearch, DynamoDB, Kafka, RabbitMQ
Cloud, DevOps & Tools: AWS (EC2, S3, Lambda, RDS), Docker, Kubernetes, Helm, Terraform, Jenkins, Git, Linux, CMake, CI/CD
Observability: Splunk, OpenTelemetry, Datadog, Prometheus, Grafana, ELK Stack, Jaeger